

Éléments de théorie pour la construction des builders. Le but de ce petit article est de donner la base de la théorie pour prouver la correction de l'algorithme qui construit les builders pointés.

Quelques éléments de définition:

- On appelle classes auxiliaires les règles correspondant aux RuleName dans la bibliothèque UsainBoltz.
- On appelle constructeurs ce qui est nommé builder au sein de la bibliothèque UsainBoltz.
- On note $|t|_z$ la taille d'un tuple t en tant qu'objet d'une classe combinatoire.

Quelques éléments de simplification par rapport aux vrais algorithmes point et unpoint:

- on considère qu'il existe une règle *Zero* qui correspond à la classe vide, cette classe est utilisée en lieu et place des simplifications qui ont place dans le vrai algorithme Point.

Commençons par définir le système de type dans lequel nous allons travailler, un objet de type α est

$$\begin{aligned}
\alpha &= Atom \\
&| Epsilon \\
&| Marker(string) \\
&| (1, \alpha) \cup \dots \cup (n, \alpha) & \forall n \in \mathbb{N}^* \\
&| \alpha \times \dots \times \alpha \\
&| List \alpha \\
&| Built(Pas mon probleme)
\end{aligned}$$

Un constructeur est de type: $\alpha \rightarrow \alpha$.

Le générateur d'Usainboltz fonctionne à l'aide d'un algorithme que l'on note *B*, cet algorithme prend en argument une règle et en déduit un constructeur pour cette règle. Il est possible de décrire le comportement de cet algorithme en donnant le type des constructeurs renvoyés. Pour ce faire, on s'aide d'un ensemble de règles d'inférences de types que l'on décrit ici.

Definition 1 (Tuple acceptable pour une règle.) *Le contexte noté Γ est la donnée d'une grammaire et un ensemble de constructeurs: $\alpha \rightarrow Built()$, chaque constructeur est associé à une classe auxiliaire de manière à ce que chaque classe auxiliaire soit associée à un constructeur.*

Pour un contexte Γ , pour A une règle de la grammaire de Γ , t un type de constructeur, $B(A)$ est de type t pour Γ et on note $\Gamma : B(A) \vdash t$ si cette proposition peut être déduite des règles d'inférences que nous allons lister.

$$\begin{array}{c}
\frac{}{\Gamma \vdash B(Epsilon()) : \textit{Epsilon} \rightarrow \textit{Epsilon}} \textit{Epsilon} \\
\\
\frac{}{\Gamma \vdash B(Atom()) : \textit{Atom} \rightarrow \textit{Atom}} \textit{Atom} \\
\\
\frac{}{\Gamma \vdash B(Marker('a')) : \textit{Marker}('a') \rightarrow \textit{Marker}('a')} \textit{Marker} \\
\\
\frac{\forall i \in [1; n] \quad \Gamma \vdash \alpha_i \rightarrow \alpha}{\Gamma \vdash B(A_1 + \dots + A_n) : (1, \alpha_1) \cup \dots \cup (n, \alpha_n) \rightarrow \alpha} \textit{Union } n \geq 1 \\
\\
\frac{\Gamma \vdash B(A_1) : \alpha_1 \rightarrow \alpha'_1 \quad \dots \quad \Gamma \vdash B(A_n) : \alpha_n \rightarrow \alpha'_n}{\Gamma \vdash B(A_1 \times \dots \times A_n) : \alpha_1 \times \dots \times \alpha_n \rightarrow \alpha'_1 \times \dots \times \alpha'_n} \textit{Product } n \geq 2 \\
\\
\frac{G : A \vdash t_1 \quad \dots \quad G : A \vdash t_n}{G : \textit{Seq}(A) \vdash (t_1, \dots, t_n)} \textit{Sequence} \\
\\
\frac{A \text{ est une classe auxiliaire associée à la règle } R \text{ dans } G \quad G : R \vdash t}{G : A \vdash t} \textit{Simplification}
\end{array}$$

Les builders font appel à des tuples imbriqués comme arguments, la première question est donc de savoir à quelles conditions un builder peut être considéré comme correctement typé. Il faudra ensuite montrer que des tuples pointés sont envoyés par l'algorithme vers des tuples non pointés. Enfin, il faut s'assurer que n tuples pointés de taille n (en tant qu'objet d'une classe combinatoire) sont envoyés vers un seul tuple non pointé de taille n . Cela devrait suffire.

La première étape est inspirée des arbres de preuves.

À l'aide entre autre de cette théorie, il faut alors montrer les trois théorèmes suivants:

Theorem 1 (Unpoint réalise bien une opération de dépointage) Soit G , A , t une grammaire, une règle et un tuple tel que A apparaisse dans G . $G^\bullet : A^\bullet \vdash t$ si et seulement si $\textit{Unpoint}(t, A)$ est correctement défini et $G : A \vdash \textit{Unpoint}(t, A)$.

Theorem 2 (Unpoint respecte la taille des tuples) Soit G , A , t tel que $G^\bullet : A^\bullet \vdash t$. Alors $|\textit{Unpoint}(t, A)|_z = |t|_z$.

Theorem 3 (Il y a n tuples pointés pour un tuple non pointé de taille n) Soit G , A , t , tel que $G^\bullet : A^\bullet \vdash t$, alors $|\textit{Unpoint}^{-1}(\{t\}, A)| = |t|_z$.

L'idée de la preuve est la même pour les trois, une induction sur A .

On commence par prouver un petit lemme utile:

Lemme 1 Soit G , A , t une grammaire, une règle et un tuple. On a $G^\bullet : A \vdash t$ avec A une règle apparaissant dans G si et seulement si $G : A \vdash t$.

Proof 1 Dans le sens direct, l'opération de pointage fait apparaître de nouvelles règles mais n'en supprime ni n'en modifie aucune. Donc A n'a pas été modifié, donc l'arbre de preuve utilisé pour obtenir que $G^\bullet : A \vdash t$ permet de prouver $G : A \vdash t$.

On utilise le même argument dans l'autre sens.

Proof 2 (Unpoint réalise bien une opération de dépointage) Cas de bases:

- $A = \text{Epsilon}()$ donc $A^\bullet = \text{Zero}()$, on ne peut pas avoir $\text{Zero}() \vdash t$ car $\text{Zero}()$ n'est pas associé à une règle d'inférence. D'un autre coté, $\text{Unpoint}(t, \text{Epsilon}())$ lève une exception (donc n'est pas correctement défini).
- $\text{Marker}()$ et $\text{Zero}()$ sont très similaires à $\text{Epsilon}()$.
- $A = \text{Atom}()$ donc $A^\bullet = \text{Atom}()$. On a donc $\text{Unpoint}(t, A) = t$. Si $G^\bullet : \text{Atom}() \vdash t$, alors on peut utiliser la seule règle d'inférence faisant intervenir l'atome pour en déduire que $t = ('z')$. Donc, en appliquant cette même règle d'inférence on en déduit que $G : \text{Atom}() \vdash t$. Le sens inverse est identique.

Cas rékursifs:

- $A = A_1 + \dots + A_n$, alors $A^\bullet = A_1^\bullet + \dots + A_n^\bullet$. On applique l'unique règle d'inférence associée à l'union, on en déduit que $t = (i, t')$ avec $1 \leq i \leq n$ et $G^\bullet : A_i^\bullet \vdash t'$. A apparait dans G , donc A_i apparait dans G , on peut donc utiliser notre induction. Par hypothèse d'induction, $G : A_i \vdash \text{Unpoint}(t', A_i)$ et donc, en appliquant la règle d'inférence pour l'union,

$$G : A \vdash (i, \text{Unpoint}(t', A_i)) = \text{Unpoint}((i, t'), A) = \text{Unpoint}(t, A)$$

. Le sens réciproque est identique.

- $A = A_1 \times \dots \times A_n$, alors $A^\bullet = A_1^\bullet \times A_2^\bullet \times \dots \times A_n^\bullet + \dots + A_1^\bullet \times \dots \times A_{n-1}^\bullet \times A_n^\bullet$. En appliquant la règle d'inférence pour l'union, on en déduit que $t = (i, t')$ avec $t' = (t_1, \dots, t_n)$. Puisque $G^\bullet : A_1^\bullet \times \dots \times A_i^\bullet \times \dots \times A_n^\bullet \vdash (t_1, \dots, t_n)$, on peut cette fois ci appliquer la règle d'inférence pour le produit. Alors, pour $j \neq i$ $G^\bullet : A_j \vdash t_j$, or A_j apparait dans G car A apparait dans G , donc on peut appliquer notre lemme pour obtenir que $G : A_j \vdash t_j$. Par hypothèse d'induction, on sait aussi que $G : A_i \vdash \text{Unpoint}(t_i, A_i)$, on peut utiliser l'inférence pour le produit pour en déduire que

$$G : A_1 \times \dots \times A_n \vdash (t_1, \dots, \text{Unpoint}(t_i, A_i), \dots, t_n) = \text{Unpoint}(t, A)$$

Le sens réciproque est identique.

- Si A est une classe auxiliaire, la seule règle applicable est la simplification, on a donc $A^\bullet : R \in G^\bullet$, or A apparait dans G . Par définition du pointage, il existe T tel que $T^\bullet = R$ et $A : T \in G$. Par hypothèse, $G^\bullet : A^\bullet \vdash t$ donc $G^\bullet : R \vdash T$. On conclut par hypothèse d'induction et application de la règle de simplification. Le sens réciproque est identique.